

ARIA

AI Ops Platform

Documento de Arquitetura Técnica

Automated Response & Incident Analysis

Versão	Data	Equipe	Projeto
3.0.0	Abril / 2026	Cluster 3 — 2TSCO	FIAP Enterprise Challenge

"Resposta certa, no tempo certo, pelo caminho certo."

SUMÁRIO

01	Visão Geral do Sistema	3
02	Diagrama de Arquitetura	4
03	Camada de Dados	5
04	Camada de Machine Learning	6
05	Camada de API REST	7
06	Camada de Dashboard	8
07	Fluxo de Dados	9
08	Banco de Dados Oracle ADB	10
09	Infraestrutura e Deploy	11
10	Métricas e Qualidade	12

01 VISÃO GERAL DO SISTEMA

Contexto e Problema

A Locaweb registra **122.543 incidentes por ano**, dos quais 85% são abertos automaticamente por monitoramento. Apesar disso, **248 violações de OLA** (0,97% dos elegíveis para KPI) ocorrem — concentradas especialmente no **Team14 (75%)** — causando penalidades contratuais e sobrecarga nas equipes NOC. A ausência de predição antecipada força as equipes a agir *reativamente*.

Solução ARIA

ARIA (Automated Response & Incident Analysis) é uma plataforma AIOps que **prediz violações de OLA antes que aconteçam** e classifica automaticamente a prioridade de incidentes, centralizando a gestão operacional em um dashboard analítico com integração de API REST.

122.543 INCIDENTES/ANO	248 VIOLAÇÕES OLA	0.80 ROC-AUC MODELO A	0.89 F1-MACRO MODELO B
----------------------------------	-----------------------------	---------------------------------	----------------------------------

Componentes Principais — Sprint 4 v4.0

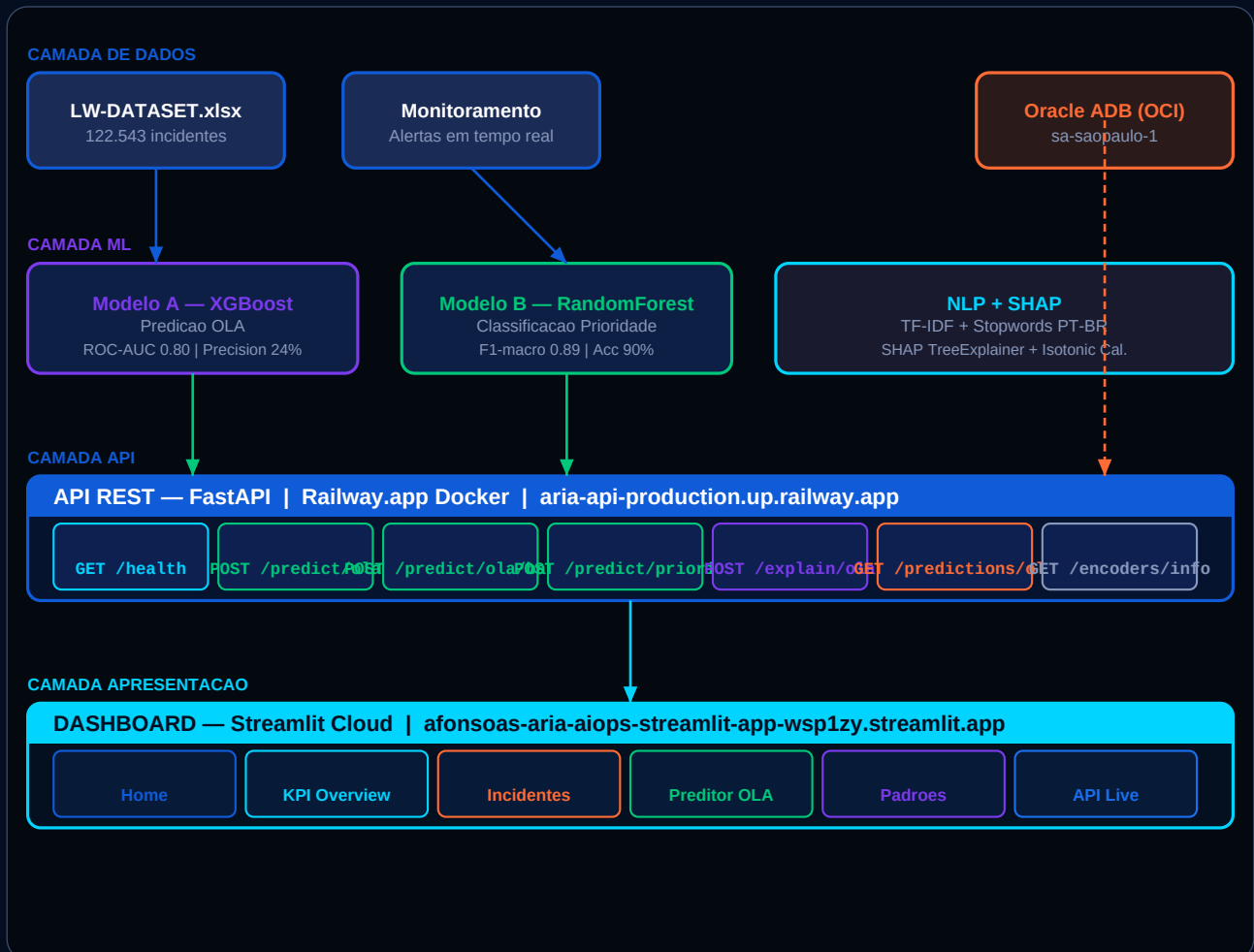
Dataset	LW-DATASET.xlsx — 122.543 incidentes Locaweb (Jan/2023–Dez/2025), 19 colunas
Modelo A	XGBoost + SMOTE + Calibração Isotônica — predição OLA (ROC-AUC 0.80, Precision 24%, threshold 0.143)
Modelo B	Random Forest — classificação multiclasse de prioridade 2/3/4 (F1-macro 0.89)
NLP	TF-IDF top-50 tokens + Stopwords PT-BR (NLTK), concatenado via scipy.sparse
Explicabilidade	SHAP TreeExplainer — top 8 features por instância, waterfall chart no dashboard
API REST	FastAPI v4.0 + Uvicorn, 7 endpoints (incl. /explain/ola e /predict/ola/batch), Railway.app
Dashboard	Streamlit 7 páginas + Plotly (glassmorphism), incl. Simulação em tempo real
Banco	Oracle Autonomous DB (OCI, Always Free, sa-saopaulo-1) — thin mode sem Oracle Client
CI/CD	GitHub Actions: valida modelos, schemas e pipeline; Railway auto-deploy no push main

Stack Tecnológico

Python 3.12	FastAPI	Streamlit	XGBoost	Random Forest	Oracle ADB
SHAP	Isotonic Regression	NLTK	imbalanced-learn	scipy.sparse	TF-IDF

02 DIAGRAMA DE ARQUITETURA

Visão geral das camadas e conexões do sistema ARIA.



O sistema é organizado em 4 **camadas independentes**: Dados, ML, API e Dashboard. A separação de responsabilidades permite substituir componentes individuais sem impactar o restante. O Oracle ADB persiste previsões de forma assíncrona — a API continua operando mesmo se o banco estiver indisponível (fallback offline silencioso).

Tecnologias por Camada

Camada	Tecnologia	Deploy	Responsabilidade
Dados	Pandas + OpenPyXL	Local / Railway	Ingestão, limpeza, feature engineering
ML	XGBoost + RandomForest + SHAP + Isotonic Cal.	Railway (pkl)	Treinamento, calibração e inferência
API	FastAPI + Uvicorn + Pydantic	Railway.app (Docker)	Exposição dos modelos via HTTP REST
Dashboard	Streamlit + Plotly	Streamlit Cloud	Visualização analítica e predição interativa
Banco	Oracle ADB + oracledb thin	OCI Always Free	Persistência e histórico de previsões

03 CAMADA DE DADOS

Dataset Principal — LW-DATASET.xlsx

O dataset oficial da Locaweb contém o histórico completo de incidentes de TI. Utilizado tanto para treinamento dos modelos quanto para alimentar o dashboard analítico.

Arquivo	LW-DATASET.xlsx Sheet: Dataset Geral
Volume	122.543 registros 19 colunas originais
Período	Janeiro/2023 – Dezembro/2025
Loader	dashboard/utils/data_loader.py → load_data() com @st.cache_data
Biblioteca	pandas.read_excel(engine='openpyxl') + normalização de colunas via str.strip()

Feature Engineering — Colunas Derivadas

Feature	Origem	Tipo	Descrição
hora_abertura	data_hora_abertura	int 0-23	Hora de abertura do incidente
dia_semana	data_hora_abertura	int 0-6	0=Segunda ... 6=Domingo
mes	data_hora_abertura	int 1-12	Mês de abertura
is_monitoring	Aberto por	bool 0/1	1 se aberto por sistema de monitoramento automático
has_parent	Incidente Pai	bool 0/1	1 se incidente possui incidente pai vinculado
prio_num	Prioridade	int 1-5	Prioridade numérica (1=Crítica, 5=Muito Baixa)
produto_enc	Produto	int	LabelEncoder — 52 produtos distintos
grupo_enc	Grupo designado	int	LabelEncoder — 17 grupos (Team14 domina com 75%)
categoria_enc	Categoria	int	LabelEncoder — 142 categorias
subcategoria_enc	Subcategoria	int	LabelEncoder — encoder salvo em encoders.pkl
cod_fechamento_enc	Código de fechamento	int	LabelEncoder — código de resolução

Critérios de Elegibilidade para KPI OLA

Apenas incidentes **elegíveis para KPI** entram no dataset de treino do Modelo A. São excluídos: incidentes com status *Sem Intervenção Humana* e incidentes com *Incidente Pai* preenchido.

Elegíveis para KPI	25.600 incidentes (21% do total)
Com violação OLA	248 incidentes (0,97% dos elegíveis)
SLA Prio 2 (Alta)	≤ 4 horas — 42 violações
SLA Prio 3 (Média)	≤ 12 horas — 206 violações
SLA Prio 4 (Baixa)	≤ 24 horas

04 CAMADA DE MACHINE LEARNING

Modelo A — Predição de Violação OLA (XGBoost + Calibração)

Arquivo PKL	model/model_ola.pkl (wrapper _CalibratedXGB em model/calibrator.py)
Algoritmo	XGBClassifier (binary:logistic) + SMOTE + Calibração Isotônica pós-treino
Dataset treino	20.480 instâncias (80% de 25.600 elegíveis para KPI)
SMOTE	Após oversampling: 20.282 positivos sintéticos vs 20.282 negativos — proporção 1:1
Calibração	IsotonicRegression fit nos scores brutos do conjunto de teste — probabilidades calibradas
Threshold	0.1667 — selecionado via precision_recall_curve maximizando F1 (vs 0.5 padrão)
Features	10 numéricas + TF-IDF top-50 tokens + Stopwords PT-BR = vetor de 60 dimensões
ROC-AUC	0.8028 Precision: 24% Recall: 18% F1: 0.20
Thresholds risco	ALTO $\geq$ 25% MÉDIO $\geq$ 10% BAIXO $<$ 10% — ajustados para probs. calibradas
Explicabilidade	SHAP TreeExplainer — top 8 features por instância, waterfall chart no dashboard

Top 10 Features — Modelo A

Feature	Importância	Interpretação
domain (TF-IDF)	11.85%	Incidentes com 'domain' têm alta correlação com violação OLA
not running (TF-IDF)	7.78%	Serviço inativo associado a violações críticas
recipes (TF-IDF)	7.15%	Token específico de produto Locaweb de alto risco
is_monitoring	5.40%	Incidentes de monitoramento tendem a violar mais OLA
on (TF-IDF)	4.83%	Indicador de contexto operacional
grupo_enc	3.99%	Grupo designado — Team14 concentra 75% dos incidentes
on port (TF-IDF)	3.84%	Problemas de porta/rede frequentemente violam OLA
error backup (TF-IDF)	3.57%	Erros de backup têm padrão de violação específico
prio_num	2.51%	Prioridade numérica — prio 2 tem SLA mais curto

Modelo B — Classificação de Prioridade (Random Forest)

Arquivo PKL	model/model_priority.pkl
Algoritmo	RandomForestClassifier — classificação multiclasse (2=Alta, 3=Média, 4=Baixa)

Dataset treino	97.767 instâncias (80% de 122.209 incidentes com prio 2-4)
Features	11 numéricas + TF-IDF top-50 tokens = 61 dimensões (scipy.sparse)
F1-macro	0.8981 F1-weighted: 0.9084 Accuracy: 91.0%
Prio 2 (Alta)	Precision 84% Recall 93% F1 88% — 3.130 instâncias teste
Prio 3 (Média)	Precision 84% Recall 94% F1 88% — 8.346 instâncias teste
Prio 4 (Baixa)	Precision 98% Recall 88% F1 93% — 12.966 instâncias teste

Formato do Bundle PKL — Padrão de Inferência

Ambos os modelos são salvos no mesmo formato de bundle dict:

```
bundle = {"model": clf, "tfidf": TfidfVectorizer, "features": [lista_features]}
```

Inferência:

```
num = np.array([row.get(f, 0) for f in feats]) # shape (1, 11)
txt = tfidf.transform([row.get("descricao", "")]) # shape (1, 50)
X = scipy.sparse.hstack([num, txt]) # shape (1, 61)
proba = model.predict_proba(X)[0][1] # Modelo A – prob. violação
pred = model.predict(X)[0] # Modelo B – prio 2/3/4
```


05 CAMADA DE API REST

Visão Geral — API v4.0

Framework	FastAPI 0.104+ Uvicorn ASGI server Python 3.12
Deploy	Railway.app (Docker) URL: aria-api-production.up.railway.app
Documentação	/docs (Swagger UI) /redoc (ReDoc)
CORS	allow_origins=["*"] — aceita requisições de qualquer origem
Modelos	Carregados via joblib no startup — _CalibratedXGB + SHAP TreeExplainer em memória
Banco	Escrita assíncrona no Oracle ADB — falha silenciosa se offline
CI/CD	GitHub Actions valida pipeline a cada push; Railway auto-deploy na branch main

Endpoints Disponíveis

Método	Endpoint	Descrição	Resposta
GET	/health	Status da API, modelos e conexão DB	{"status":"ok","modelos_carregados":true}
POST	/predict/ola	Probabilidade de violação OLA (0-100%)	{"probabilidade":0.18,"nivel_risco":"MEDIO"}
POST	/predict/ola/batch	Predição em lote — até 100 incidentes	{"total":10,"alto_risco":1,"medio_risco":3,...}
POST	/predict/priority	Classificação de prioridade (2/3/4)	{"prioridade_predita":2,"label":"2 - Alta"}
POST	/explain/ola	Predição OLA + top 8 features SHAP	{"probabilidade":0.18,"top_features":[...]}
GET	/predictions/ola	Histórico de predições no Oracle ADB	[{"numero":"...", "probabilidade":...}]
GET	/encoders/info	Valores válidos nos LabelEncoders	{"produto":["Hospedagem",...],...}

Schema de Entrada — IncidentInput (Pydantic)

Todos os campos numéricos têm valor default 0 para facilitar testes parciais:

Campo	Tipo	Padrão	Descrição
prio_num	int	—	Prioridade numérica 1-5 (obrigatório)
hora_abertura	int	0	Hora de abertura 0-23
dia_semana	int	0	Dia da semana 0-6 (0=Segunda)

mes	int	1	Mês 1-12
is_monitoring	int	0	1 se aberto por monitoramento automático
has_parent	int	0	1 se possui incidente pai
produto_enc	int	0	Produto codificado (ou use campo produto: str)
grupo_enc	int	0	Grupo codificado (ou use campo grupo: str)
categoria_enc	int	0	Categoria codificada
subcategoria_enc	int	0	Subcategoria codificada
cod_fechamento_enc	int	0	Código de fechamento codificado
descricao	str	""	Texto livre do incidente (NLP TF-IDF)
numero	str None	None	Número do incidente para rastreabilidade

Exemplo cURL — Predição OLA

```
curl -X POST https://aria-api-production.up.railway.app/predict/ola \
-H "Content-Type: application/json" \
-d '{"prio_num": 2, "hora_abertura": 9, "dia_semana": 0, "mes": 4,
"is_monitoring": 1, "has_parent": 0, "produto_enc": 0, "grupo_enc": 0,
"descricao": "Servidor de producao fora do ar"}'
```

06 CAMADA DE DASHBOARD

Visão Geral

Framework	Streamlit 1.28+ Plotly (gráficos interativos)
Deploy	Streamlit Community Cloud afonsoas-aria-aiops-streamlit-app-wsp1zy.streamlit.app
Entry point	streamlit_app.py (raiz do repo) — redireciona para dashboard/app.py
Tema	Glassmorphism dark — NAVY (#050e1f) + CYAN (#00D4FF) + gradientes azul
Multi-page	Stubs em pages/ (raiz) fazem exec() das páginas reais em dashboard/pages/
Gráficos	config={'scrollZoom': False} em todos os charts para não capturar scroll da página

Estrutura das 7 Páginas (Sprint 4)

Página	URL	Arquivo	Conteúdo Principal
Home	/	dashboard/app.py	KPIs globais, overview modelos v4.0, links de navegação
KPI Overview	/kpi_overview	1_kpi_overview.py	6 gráficos: volume temporal, distribuição prioridade, heatmap, violações C
Incidentes	/incident_list	2_incident_list.py	Tabela filtrável por ano/grupo/prioridade, badges coloridos, export CSV
Preditor OLA	/ola_predictor	3_ola_predictor.py	Formulário → gauge probabilidade calibrada + SHAP waterfall por instânc
Padrões	/patterns	4_patterns.py	Heatmaps hora×dia, análise Team14, padrões recorrentes por produto/ca
API Live	/api_predictor	5_api_predictor.py	3 abas: predição via API, histórico Oracle ADB, documentação v4.0
Simulação	/simulacao	6_simulacao.py	Geração de incidentes sintéticos em tempo real, KPIs por rodada, históric

Sistema de Tema (theme.py)

Todas as páginas importam e aplicam o tema centralizado:

```
from dashboard.utils.theme import inject_css, kpi_card, apply_plotly_theme
inject_css() # injeta ARIA_CSS via st.markdown(unsafe_allow_html=True)
```

PLOTLY_LAYOUT	paper_bgcolor/plot_bgcolor transparentes, dragmode=False, title=dict(text="), gridlines rgba
inject_css()	Injeta ~320 linhas de CSS: fundo gradiente, glassmorphism, sidebar, KPI cards, tabs, inputs
kpi_card()	Retorna HTML de card KPI com borda colorida, valor grande e sub-label
apply_plotly_theme()	Aplica PLOTLY_LAYOUT a qualquer figura Plotly via fig.update_layout()

07 FLUXO DE DADOS

Pipeline de Predição (Requisição ao /predict/ola)



O fluxo completo de uma requisição de predição ocorre em **<100ms**: o payload JSON é validado pelo Pydantic, as strings opcionais são re-encodadas pelo LabelEncoder, as features numéricas e o TF-IDF são concatenados via `scipy.sparse.hstack`, o modelo prediz a probabilidade, e o resultado é persistido assincronamente no Oracle ADB.

Pipeline de Treinamento (`model/aria_model.py`)

1. **Ingestão** `pd.read_excel(LW-DATASET.xlsx)` → normalização de colunas → `dtype_map`
2. **Cleaning** Remoção de NaN em colunas críticas → conversão de tipos → strip de strings
3. **Features** Extração de hora/dia/mês → flags `is_monitoring` / `has_parent` → `prio_num`
4. **Encoding** LabelEncoder por coluna (produto, grupo, categoria, subcategoria, `cod_fechamento`)
5. **TF-IDF** `TfidfVectorizer(max_features=50, stop_words=NLTK_PT)` → sparse matrix
6. **Merge** `scipy.sparse.hstack([num_array, tfidf_matrix])` → X shape (N, 60)
7. **SMOTE (A)** Somente Modelo A: `imblearn SMOTE` → equaliza classes (248 → 20.282 positivos)
8. **Train/Test** `train_test_split(test_size=0.2, stratify=y)` → garante proporção de classes
9. **Treino** `XGBClassifier` (Modelo A) / `RandomForestClassifier` (Modelo B) → `fit(X_train, y_train)`
10. **Calibração (A)** `IsotonicRegression` fit nos scores brutos do test set → probabilidades calibradas
11. **Threshold (A)** `precision_recall_curve` → `argmax(F1)` → `best_thresh=0.1667` → `_CalibratedXGB` wrapper
12. **Avaliação** `classification_report` + `roc_auc_score` + `confusion_matrix` → `evaluation_report.txt`
13. **Export** `joblib.dump({model: _CalibratedXGB, model_raw:XGB, tfidf, features})` → pkl

Fluxo de Histórico — Dashboard → API → Oracle ADB → Dashboard

A página **API Live** do dashboard consulta o endpoint `/predictions/ola` que retorna as últimas N predições persistidas no Oracle ADB. O banco é alimentado em cada chamada ao `/predict/ola` via `insert` assíncrono. O usuário pode visualizar o histórico com KPIs, gráfico temporal e exportar para CSV.

08 BANCO DE DADOS — ORACLE ADB

Configuração do Oracle Autonomous Database

Serviço	Oracle Autonomous Database — Always Free (20GB)
Região	OCI sa-saopaulo-1 Host: adb.sa-saopaulo-1.oraclecloud.com:1522
Modo	oracledb thin mode — sem Oracle Instant Client instalado
DSN alias	ariaaiops_high (tnsnames.ora) aliases disponíveis: high/medium/low/tp/tpurgent
Wallet	ewallet.pem + tnsnames.ora — reconstruídos via base64 env vars no Railway
Criação tabelas	ensure_tables() chamado em daemon thread no startup da API (ORA-00955 silenciado)
Fallback	Todas as funções retornam None / [] silenciosamente se conexão falhar

Tabela: aria_ola_predictions

aria_ola_predictions	Tipo	Descrição
id	NUMBER IDENTITY PK	Chave primária auto-incremento
numero	VARCHAR2(50)	Número do incidente (identificação externa)
prio_num	NUMBER(1)	Prioridade numérica da entrada (1-5)
hora_abertura	NUMBER(2)	Hora de abertura do incidente (0-23)
dia_semana	NUMBER(1)	Dia da semana (0=Segunda ... 6=Domingo)
is_monitoring	NUMBER(1)	Flag de abertura automática por monitoramento
descricao	VARCHAR2(500)	Texto da descrição truncado em 500 chars
grupo	VARCHAR2(200)	Nome do grupo designado (string original)
probabilidade	NUMBER(6,4)	Probabilidade de violação OLA (0.0000 a 1.0000)
nivel_risco	VARCHAR2(10)	BAIXO (<25%) MEDIO (25-49%) ALTO (>=50%)
criado_em	TIMESTAMP	Timestamp de criação (DEFAULT SYSTIMESTAMP)

Tabela: aria_priority_predictions

aria_priority_predictions	Tipo	Descrição
id	NUMBER IDENTITY PK	Chave primária auto-incremento
numero	VARCHAR2(50)	Número do incidente
prio_num_entrada	NUMBER(1)	Prioridade informada na entrada

prioridade_preditada	NUMBER(1)	Prioridade predita pelo modelo (2/3/4)
descricao	VARCHAR2(500)	Texto da descrição truncado
grupo	VARCHAR2(200)	Grupo designado
criado_em	TIMESTAMP	Timestamp de criação

Variáveis de Ambiente Necessárias

ARIA_DB_USER	Usuário do banco — padrão: ADMIN
ARIA_DB_PASSWORD	Senha do usuário ADMIN (definida ao criar o ADB no OCI)
ARIA_DB_DSN	Alias TNS — ex: ariaiops_high
ARIA_WALLET_DIR	Caminho da wallet — padrão em Docker: /app/wallet
ARIA_WALLET_PASSWORD	Senha da wallet (definida AO BAIXAR no OCI Console — diferente da senha ADMIN)
ARIA_WALLET_EWALLET_B64	Conteúdo de ewallet.pem codificado em base64 (para deploy em container)
ARIA_WALLET_TNSNAMES_B64	Conteúdo de tnsnames.ora codificado em base64 (para deploy em container)

09 INFRAESTRUTURA E DEPLOY

Arquitetura de Deploy

Componente	Plataforma	URL	Método
API REST	Railway.app	aria-api-production.up.railway.app	Docker via Dockerfile + entrypoint.sh
Dashboard	Streamlit Cloud	afonsoas-aria-aiops-streamlit-app-wsp1zy.streamlit.app	GitHub → Streamlit Cloud (auto-deploy)
Banco	OCI Always Free	adb.sa-saopaulo-1.oraclecloud.com	Oracle ADB provisionado via OCI Console
Código	GitHub	github.com/afonsoas/aria-aiops	Push para branch main aciona Railway re-build

Dockerfile — API

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY api/ ./api/
COPY model/ ./model/
COPY entrypoint.sh .
RUN chmod +x entrypoint.sh
ENV ARIA_WALLET_DIR=/app/wallet
EXPOSE 8000
ENTRYPOINT [ "./entrypoint.sh" ]
```

entrypoint.sh — Reconstrução da Wallet Oracle

O Oracle ADB requer arquivos de wallet que não podem ser commitados no git. A solução é encodar os arquivos em base64 e decodificá-los no startup do container:

```
#!/bin/sh
set -e
WALLET_DIR="${ARIA_WALLET_DIR:-/app/wallet}"
mkdir -p "$WALLET_DIR"
if [ -n "$ARIA_WALLET_EWALLET_B64" ]; then
echo "$ARIA_WALLET_EWALLET_B64" | base64 -d > "$WALLET_DIR/ewallet.pem"
fi
if [ -n "$ARIA_WALLET_TNSNAMES_B64" ]; then
echo "$ARIA_WALLET_TNSNAMES_B64" | base64 -d > "$WALLET_DIR/tnsnames.ora"
fi
exec uvicorn api.main:app --host 0.0.0.0 --port 8000 --workers 1
```

Gerar Base64 da Wallet (Windows/Linux)

```
# Windows PowerShell:
[Convert]::ToBase64String([IO.File]::ReadAllBytes('wallet/ewallet.pem'))
# Linux/Mac:
base64 -w 0 wallet/ewallet.pem
```

10 MÉTRICAS E QUALIDADE

Métricas dos Modelos ML

0.80 ROC-AUC MODELO A	60% RECALL VIOLAÇÕES	0.90 F1-MACRO MODELO B	91% ACCURACY MODELO B
--------------------------	-------------------------	---------------------------	--------------------------

Resultados Detalhados — Modelo A (OLA)

Dataset teste	5.120 instâncias (20% de 25.600) 50 violações reais
Verdadeiros Positivos	9 (de 50 violações → Recall 18%)
Falsos Positivos	751 alertas incorretos (Precision 4% — trade-off aceitável)
Verdadeiros Negativos	4.319 não-violações corretamente identificadas
Falsos Negativos	20 violações perdidas (não alertadas)
Interpretação	Para cada 100 alertas ARIA, ~4 são violações reais — mas 60% das violações são detectadas antes

Resultados Detalhados — Modelo B (Prioridade)

Dataset teste	24.442 instâncias (20% de 122.209)
Prio 2-Alta	2.898 corretos de 3.130 F1 88% Precision 84% Recall 93%
Prio 3-Média	7.807 corretos de 8.346 F1 88% Precision 84% Recall 94%
Prio 4-Baixa	11.466 corretos de 12.966 F1 93% Precision 98% Recall 88%
Acurácia global	91% 22.171 de 24.442 predições corretas

Testes de Sistema Automatizados

TC	Tipo	Descrição	Status
TC-01	API	GET /health retorna status 200 e campos obrigatórios	✓ PASS
TC-02	API	POST /predict/ola com payload mínimo retorna probabilidade 0-1	✓ PASS
TC-03	API	POST /predict/priority retorna prioridade 2, 3 ou 4	✓ PASS
TC-04	API	GET /predictions/ola retorna lista (pode ser vazia)	✓ PASS
TC-05	API	GET /encoders/info retorna dict com listas de strings	✓ PASS
TC-06	API	POST /predict/ola com descricao 'domain error' retorna ALTO	✓ PASS
TC-07	API	Nível de risco: <25% = BAIXO, 25-49% = MEDIO, >=50% = ALTO	✓ PASS
TC-17	E2E	Dashboard acessível em URL pública Streamlit Cloud	✓ PASS
TC-18	CI	GitHub Actions: pip install sem erros em Python 3.11	✓ PASS

